Ex:No: 1A Date:	**Billing System Calculation Using Arithmetic Expressions**

Problem Description:

A **store owner** wants to calculate the **total cost, discount, and final bill** for a customer. The store offers a **10% discount** if the total purchase exceeds 500 units. Write an R program that takes the **price of an item** and **quantity purchased** as input and calculates:

1. Total cost
2. Discount (if applicable)
3. Final bill amount.

Objective:

To calculate the total cost, applicable discount, and final bill amount using arithmetic expressions and conditional statements in R, demonstrating how decision-making logic can be applied in a real-world billing system scenario.

Sample Input	Expected Output
Enter price of item: 250 Enter quantity purchased: 3	Total cost: 750 Discount: 75 Final bill amount: 675
Enter price of item: 100 Enter quantity purchased: 5	Total cost: 500 Discount: 0 Final bill amount: 500

Program:

```r
# Taking input from user
price <- as.numeric(readline(prompt = "Enter price of item: "))
quantity <- as.numeric(readline(prompt = "Enter quantity purchased: "))

# Calculating total cost
total_cost <- price * quantity

# Applying discount condition
if (total_cost > 500) {
  discount <- total_cost * 0.10
} else {
  discount <- 0
}

# Calculating final bill
final_bill <- total_cost - discount

# Displaying results
cat("Total cost:", total_cost, "\n")
cat("Discount:", discount, "\n")
cat("Final bill amount:", final_bill, "\n")
```

```
> source("~/.active-rstudio-document")
Enter price of item: 250
Enter quantity purchased: 3
Total cost: 750
Discount: 75
Final bill amount: 675
>
```

Ex:No: 1B Date:	**Grade Evaluation and Feedback Using Control Statements**

Problem Description:

A teacher wants to evaluate students' marks and assign grades based on their score:

- Marks 90 and above → Grade A
- Marks 75 to 89 → Grade B
- Marks 50 to 74 → Grade C
- Marks below 50 → Grade F

Additionally, the program should provide **feedback** for each grade:

- A → "Excellent performance"
- B → "Good performance"
- C → "Average performance"
- F → "Needs improvement"

Write an R program that takes the **marks of a student** as input, assigns the **appropriate grade** using if and if-else, and provides **feedback** using a switch statement.

Objective:

To evaluate student marks and assign grades with corresponding feedback using if, if-else, and switch statements in R, demonstrating conditional decision-making in a practical scenario.

Sample Input	Expected Output
Enter student marks: 80	Grade assigned: B Feedback: Good performance
Enter student marks: 50	Grade assigned: C Feedback: Average performance

Program:

```r
# Grade Evaluation Using Control Statements

# Take input marks
marks <- as.numeric(readline(prompt = "Enter student marks: "))

# Using if-else to assign grade
if (marks >= 90) {
  grade <- "A"
} else if (marks >= 75) {
  grade <- "B"
} else if (marks >= 50) {
  grade <- "C"
} else {
  grade <- "F"
}

# Display grade
cat("Grade assigned:", grade, "\n")

# Using switch statement for feedback
feedback <- switch(grade,
                   "A" = "Excellent performance",
                   "B" = "Good performance",
                   "C" = "Average performance",
                   "F" = "Needs improvement")

# Display feedback
cat("Feedback:", feedback, "\n")
```

28:33 (Top Level)

Console Terminal Background Jobs

R 4.5.3 · ~/

```
Enter student marks: 80
Grade assigned: B
Feedback: Good performance
>
```

Ex:No: 1C Date:	Fibonacci Staircase Pattern Using Loops

Problem Description:

A math teacher wants to print a **staircase pattern using Fibonacci numbers**. Each row of the staircase should contain Fibonacci numbers in sequence.

For example, if N = 5, the pattern should be:

0
0 1
0 1 1
0 1 1 2
0 1 1 2 3

Write an R program that takes N (number of rows) as input and prints the Fibonacci staircase using:

1. for loop
2. while loop

Objective:

To demonstrate the use of loops in R to generate a Fibonacci series in a structured staircase pattern, combining iterative computation with nested loops for practical pattern printing.

Sample Input	Expected Output
Enter the number of rows: 6	Fibonacci Staircase using for loop: 0 0 1 0 1 1 0 1 1 2 0 1 1 2 3 0 1 1 2 3 5 Fibonacci Staircase using while loop: 0 0 1 0 1 1 0 1 1 2 0 1 1 2 3 0 1 1 2 3 5

Program:

```r
fib <- function(n) {
  if (n == 1) {
    return(0)
  } else if (n == 2) {
    return(1)
  } else {
    a <- 0
    b <- 1
    for (i in 3:n) {
      temp <- a + b
      a <- b
      b <- temp
    }
    return(b)
  }
}
# User input
n <- as.numeric(readline(prompt = "Enter the number of rows: "))

# Using for loop
cat("Fibonacci Staircase using for loop:\n")
for (i in 1:n) {
  for (j in 1:i) {
    cat(fib(j), " ")
  }
  cat("\n")
}
# Using while loop
cat("Fibonacci Staircase using while loop:\n")
i <- 1
while (i <= n) {
  j <- 1
  while (j <= i) {
    cat(fib(j), " ")
    j <- j + 1
  }
  cat("\n")
  i <- i + 1
}
```

Console Terminal × Background Jobs ×

R 4.5.3 · ~/

```
> source("~/.active-rstudio-document")
Enter the number of rows: 6
Fibonacci Staircase using for loop:
0
0  1
0  1  1
0  1  1  2
0  1  1  2  3
0  1  1  2  3  5
Fibonacci Staircase using while loop:
0
0  1
0  1  1
0  1  1  2
0  1  1  2  3
0  1  1  2  3  5
>
```

Ex:No: 1D Date:	Password Strength Analysis Using String Operations

Problem Description:

An online system requires users to create secure passwords. An R program is needed to analyze a user-entered password by checking its length, converting case, and identifying whether it contains numbers and special characters using string operations.

Objective:

To demonstrate the application of string manipulation functions in R for analyzing and validating password strength.

Sample Input	Expected Output
Enter password: Pass@123	Password Length: 8 Uppercase Version: PASS@123 Contains Number: TRUE Contains Special Character: TRUE
Enter password: Pass	Password Length: 4 Uppercase Version: PASS Contains Number: FALSE Contains Special Character: FALSE

Program:

```r
# Password strength analysis using strings in R

# User input password
password <- readline(prompt = "Enter password: ")

# Length of password
length_pass <- nchar(password)

# Case conversion
upper_pass <- toupper(password)

# Check for number (digits)
contains_number <- grepl("[0-9]", password)

# Check for special character
contains_special <- grepl("[^A-Za-z0-9]", password)

# Display results
cat("Password Length:", length_pass, "\n")
cat("Uppercase Version:", upper_pass, "\n")
cat("Contains Number:", contains_number, "\n")
cat("Contains Special Character:", contains_special, "\n")
```

22:59 (Top Level)

Console Terminal Background Jobs

R 4.5.3 · ~/

```
Enter password: Pass@123
Password Length: 8
Uppercase Version: PASS@123
Contains Number: TRUE
Contains Special Character: TRUE
>
```

Ex:No: 1E Date:	**Monthly Sales Analysis Using Vectors**

Problem Description:

A retail store records its monthly sales figures in a vector. Write an R program to analyze the sales data by calculating the total sales, average monthly sales, highest sales value, lowest sales value, and identifying months with sales above the average.

Objective:

To apply vector operations in R for storing and analyzing numerical business data such as monthly sales figures.

Sample Input	Expected Output
Enter number of months: 5 Enter sales for month 1 : 20000 Enter sales for month 2 : 22000 Enter sales for month 3 : 19000 Enter sales for month 4 : 19500 Enter sales for month 5 : 21000	Sales Vector: 20000 22000 19000 19500 21000 Total Sales: 101500 Average Sales: 20300 Highest Sales: 22000 Lowest Sales: 19000 Above Average Sales: 22000 21000

Program:

```r
# Monthly Sales Analysis using Vectors

# User input n (number of months)
n <- as.numeric(readline(prompt = "Enter number of months: "))

# Initialize empty vector
sales <- c()

# Input sales values
for (i in 1:n) {
  value <- as.numeric(readline(prompt = paste("Enter sales for month", i, ": ")))
  sales <- c(sales, value)
}

# Display vector
cat("Sales Vector:", sales, "\n")

# Calculations
total_sales <- sum(sales)
avg_sales <- mean(sales)
highest_sales <- max(sales)
lowest_sales <- min(sales)

# Above average sales
above_avg <- sales[sales > avg_sales]

# Display results
cat("Total Sales:", total_sales, "\n")
cat("Average Sales:", avg_sales, "\n")
cat("Highest Sales:", highest_sales, "\n")
cat("Lowest Sales:", lowest_sales, "\n")
cat("Above Average Sales:", above_avg, "\n")
```

```
> source("~/.active-rstudio-document")
Enter number of months: 5
Enter sales for month 1 : 20000
Enter sales for month 2 : 22000
Enter sales for month 3 : 19000
Enter sales for month 4 : 19500
Enter sales for month 5 : 21000
Sales Vector: 20000 22000 19000 19500 21000
Total Sales: 101500
Average Sales: 20300
Highest Sales: 22000
Lowest Sales: 19000
Above Average Sales: 22000 21000
>
```

Ex:No: 1F Date:	**Household Electricity Consumption Analysis Using Matrix**

Problem Description:

An electricity board records monthly electricity consumption (in units) of multiple households. The data is stored in a matrix where **rows represent households** and **columns represent months**. Write an R program to analyze electricity usage by calculating row-wise total consumption, column-wise average consumption, highest usage, and lowest usage.

Objective:

To demonstrate the use of matrices in R for storing and analyzing two-dimensional numerical data such as electricity consumption records.

Sample Input	Expected Output
Enter number of households: 3 Enter number of months: 3 Enter electricity units: Household 1 Month 1 : 250 Household 1 Month 2 : 225 Household 1 Month 3 : 150 Household 2 Month 1 : 300 Household 2 Month 2 : 321 Household 2 Month 3 : 212 Household 3 Month 1 : 112 Household 3 Month 2 : 142 Household 3 Month 3 : 155	Electricity Consumption Matrix: Jan Feb Mar Household1 250 225 150 Household2 300 321 212 Household3 112 142 155 Total Consumption per Household: Household1 Household2 Household3 625 833 409 Average Consumption per Month: Jan Feb Mar 220.6667 229.3333 172.3333 Highest Consumption: 321 Lowest Consumption: 112

Program:

```r
h <- as.numeric(readline(prompt = "Enter number of households: "))
m <- as.numeric(readline(prompt = "Enter number of months: "))
units <- matrix(0, nrow = h, ncol = m)
for (i in 1:h) {
  for (j in 1:m) {
    units[i, j] <- as.numeric(readline(
      prompt = paste("Household", i, "Month", j, ": ")
    ))
  }
}

# Assign row and column names
rownames(units) <- paste("Household", 1:h)
colnames(units) <- paste("Month", 1:m)

# Display matrix
cat("\nElectricity Consumption Matrix:\n")
print(units)

# Row-wise total consumption
row_total <- rowSums(units)

# Column-wise average consumption
col_avg <- colMeans(units)

# Maximum and minimum consumption
max_usage <- max(units)
min_usage <- min(units)

# Display results
cat("\nRow-wise Total Consumption:\n")
print(row_total)

cat("\nColumn-wise Average Consumption:\n")
print(col_avg)

cat("\nHighest Consumption:", max_usage, "\n")
cat("Lowest Consumption:", min_usage, "\n")
```

```
Enter number of households: 3
Enter number of months: 3
Household 1 Month 1 : 250
Household 1 Month 2 : 225
Household 1 Month 3 : 150
Household 2 Month 1 : 300
Household 2 Month 2 : 321
Household 2 Month 3 : 212
Household 3 Month 1 : 112
Household 3 Month 2 : 142
Household 3 Month 3 : 155

Electricity Consumption Matrix:
             Month 1 Month 2 Month 3
Household 1      250      225      150
Household 2      300      321      212
Household 3      112      142      155

Row-wise Total Consumption:
Household 1 Household 2 Household 3
        625         833         409

Column-wise Average Consumption:
 Month 1  Month 2  Month 3
220.6667 229.3333 172.3333

Highest Consumption: 321
Lowest Consumption: 112
```

Ex:No: 1G Date:	Marks Tracking and Threshold Identification Using Multidimensional Array

Problem Description:

A college maintains the **marks of students across multiple assessments and months** for academic monitoring. The marks of **students over several days, assessments, and months** are recorded. The college wants to **store this data in a multidimensional array** (Students × Days × Assessments × Months) and **identify marks below a user-defined threshold** for remedial action.

Objective:

To apply multidimensional array operations in R for storing and analyzing student marks across multiple days, assessments, and months, and to identify low-scoring records based on a user-defined threshold.

Sample Input

Enter number of students: 2
Enter number of days per assessment: 2
Enter number of assessments: 2
Enter number of months: 2
Enter minimum marks threshold: 50

Expected Output

```
--- Month 1 ---

Enter marks for Assessment 1 :
Enter marks for Student 1 for 2 days separated by space:
1: 56 64
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 45 52
Read 2 items

Enter marks for Assessment 2 :
Enter marks for Student 1 for 2 days separated by space:
1: 75 40
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 65 38
Read 2 items
--- Month 2 ---

Enter marks for Assessment 1 :
Enter marks for Student 1 for 2 days separated by space:
1: 65 85
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 45 75
Read 2 items

Enter marks for Assessment 2 :
Enter marks for Student 1 for 2 days separated by space:
1: 56 47
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 50 56
Read 2 items
```

```
Marks Array (4-D):
, , Assessment 1, Month 1

          Day 1 Day 2
Student 1    56    45
Student 2    64    52

, , Assessment 2, Month 1

          Day 1 Day 2
Student 1    75    65
Student 2    40    38

, , Assessment 1, Month 2

          Day 1 Day 2
Student 1    65    45
Student 2    85    75

, , Assessment 2, Month 2

          Day 1 Day 2
Student 1    56    50
Student 2    47    56
```

```
Low Marks Records (< 50 ) for Month 1 :

Assessment 1 :
Student 1 Day 2 : 45

Assessment 2 :
Student 2 Day 1 : 40
Student 2 Day 2 : 38

Low Marks Records (< 50 ) for Month 2 :

Assessment 1 :
Student 1 Day 2 : 45

Assessment 2 :
Student 2 Day 1 : 47
```

Program:

```r
# Marks Tracking using Multidimensional Array (Formatted Input)

# User input for dimensions
students <- as.numeric(readline(prompt = "Enter number of students: "))
days <- as.numeric(readline(prompt = "Enter number of days per assessment: "))
assessments <- as.numeric(readline(prompt = "Enter number of assessments: "))
months <- as.numeric(readline(prompt = "Enter number of months: "))
threshold <- as.numeric(readline(prompt = "Enter minimum marks threshold: "))

# Empty vector
marks_vec <- c()

# Input marks
for (m in 1:months) {
  cat("\n--- Month", m, "---\n")

  for (a in 1:assessments) {
    cat("\nEnter marks for Assessment", a, ":\n")

    for (s in 1:students) {
      cat("Enter marks for Student", s, "for", days, "days separated by space:\n")
      input <- scan()   # FIX: removed prompt argument

      marks_vec <- c(marks_vec, input)
    }
  }
}

# Create 4D array
marks_array <- array(marks_vec,
                     dim = c(students, days, assessments, months),
                     dimnames = list(
                       paste("Student", 1:students),
                       paste("Day", 1:days),
                       paste("Assessment", 1:assessments),
                       paste("Month", 1:months)
                     ))

# Display array
cat("\nMarks Array (4-D):\n")
print(marks_array)

# Identify low marks
for (m in 1:months) {
  cat("\nLow Marks Records (<", threshold, ") for Month", m, ":\n")

  found <- FALSE

  for (a in 1:assessments) {
    for (s in 1:students) {
      for (d in 1:days) {
        mark <- marks_array[s, d, a, m]

        if (mark < threshold) {
          found <- TRUE
          cat("Assessment", a, ":\n")
          cat("Student", s, "Day", d, ":", mark, "\n")
        }
      }
    }
  }

  if (!found) {
    cat("No low marks found\n")
  }
}
```

```
Enter number of students: 2
Enter number of days per assessment: 2
Enter number of assessments: 2
Enter number of months: 2
Enter minimum marks threshold: 50

--- Month 1 ---

Enter marks for Assessment 1 :
Enter marks for Student 1 for 2 days separated by space:
1: 56 64
3:
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 45 52
3:
Read 2 items

Enter marks for Assessment 2 :
Enter marks for Student 1 for 2 days separated by space:
1: 75 40
3:
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 65 38
3:
Read 2 items

--- Month 2 ---

Enter marks for Assessment 1 :
Enter marks for Student 1 for 2 days separated by space:
1: 65 85
3:
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 45 75
3:
```

```
Enter marks for Assessment 2 :
Enter marks for Student 1 for 2 days separated by space:
1: 56 47
3:
Read 2 items
Enter marks for Student 2 for 2 days separated by space:
1: 50 56
3:
Read 2 items

Marks Array (4-D):
, , Assessment 1, Month 1

          Day 1 Day 2
Student 1    56    45
Student 2    64    52

, , Assessment 2, Month 1

          Day 1 Day 2
Student 1    75    65
Student 2    40    38

, , Assessment 1, Month 2

          Day 1 Day 2
Student 1    65    45
Student 2    85    75

, , Assessment 2, Month 2

          Day 1 Day 2
Student 1    56    50
Student 2    47    56
```

```
Low Marks Records (< 50 ) for Month 1 :
Assessment 1 :
Student 1 Day 2 : 45
Assessment 2 :
Student 2 Day 1 : 40
Assessment 2 :
Student 2 Day 2 : 38

Low Marks Records (< 50 ) for Month 2 :
Assessment 1 :
Student 1 Day 2 : 45
Assessment 2 :
Student 2 Day 1 : 47
>
```

Ex:No: 1H **Date:**	**Travel Itinerary Management Using Lists**

Problem Description:

A travel agency wants to maintain **custom travel itineraries for multiple clients**. Each itinerary includes:

- **Client Name** (string)
- **Itinerary ID** (string)
- **Destinations** (vector of city names)
- **Days Spent** (numeric vector, corresponding to each destination)
- **Mode of Transport** (vector of strings: Flight, Train, Bus)

The agency wants to:

1. **Store each client's itinerary as a list**, allowing heterogeneous data.
2. **Calculate total travel days** for each client.
3. Identify clients whose **total travel exceeds a user-defined limit** for special assistance.

This allows the agency to manage **multiple itineraries dynamically** and perform **quick analyses** on travel plans.

Objective:

To use lists in R to store heterogeneous travel itinerary data, calculate total travel days, and identify clients with extended trips for special planning or assistance.

Sample Input	
Enter number of clients: 2 Enter maximum recommended travel days: 10 Enter details for Client 1 : Enter client name: Des1 Enter itinerary ID: A123 Enter number of destinations: 3 Enter destination 1 name: Des1 Enter number of days in Des1 : 6 Enter mode of transport to Des1 : Flight Enter destination 2 name: Des2 Enter number of days in Des2 : 3 Enter mode of transport to Des2 : Car Enter destination 3 name: Des3 Enter number of days in Des3 : 2 Enter mode of transport to Des3 : Car	Enter details for Client 2 : Enter client name: Aaron Enter itinerary ID: A321 Enter number of destinations: 2 Enter destination 1 name: Des4 Enter number of days in Des4 : 4 Enter mode of transport to Des4 : Flight Enter destination 2 name: Des5 Enter number of days in Des5 : 5 Enter mode of transport to Des5 : Car

```r
# Travel Itinerary Management Using Lists

# User input
num_clients <- as.numeric(readline(prompt = "Enter number of clients: "))
max_days <- as.numeric(readline(prompt = "Enter maximum recommended travel days: "))

# Initialize list
itineraries <- list()

# Input client itineraries
for (i in 1:num_clients) {
  cat("\nEnter details for Client", i, ":\n")

  client_name <- readline(prompt = "Enter client name: ")
  itinerary_id <- readline(prompt = "Enter itinerary ID: ")
  num_dest <- as.numeric(readline(prompt = "Enter number of destinations: "))

  destinations <- c()
  days <- c()
  transport <- c()

  for (j in 1:num_dest) {
    dest <- readline(prompt = paste("Enter destination", j, "name: "))
    d <- as.numeric(readline(prompt = paste("Enter number of days in", dest, ": ")))
    t <- readline(prompt = paste("Enter mode of transport to", dest, ": "))

    destinations <- c(destinations, dest)
    days <- c(days, d)
    transport <- c(transport, t)
  }

  # Store as list
  itineraries[[i]] <- list(
    Client = client_name,
    ID = itinerary_id,
    Destinations = destinations,
    Days = days,
    Transport = transport
  )
}
```

```r
# Display all itineraries
cat("\nTravel Itineraries:\n")
print(itineraries)

# Identify clients exceeding travel limit
cat("\nClients with total travel days exceeding", max_days, ":\n")

found <- FALSE

for (i in 1:num_clients) {
  total_days <- sum(itineraries[[i]]$Days)

  if (total_days > max_days) {
    found <- TRUE
    cat("Client:", itineraries[[i]]$Client,
        "| Itinerary ID:", itineraries[[i]]$ID,
        "| Total Days:", total_days, "\n")
  }
}

if (!found) {
  cat("No clients exceed the travel limit\n")
}
```

```
> source("~/.active-rstudio-document")
Enter number of clients: 1
Enter maximum recommended travel days: 10

Enter details for Client 1 :
Enter client name: des 1
Enter itinerary ID: a123
Enter number of destinations: 3
Enter destination 1 name: des1
Enter number of days in des1 : 6
Enter mode of transport to des1 : flight
Enter destination 2 name: des2
Enter number of days in des2 : 3
Enter mode of transport to des2 : car
Enter destination 3 name: des3
Enter number of days in des3 : 2
Enter mode of transport to des3 : car

Travel Itineraries:
[[1]]
[[1]]$Client
[1] "des 1"

[[1]]$ID
[1] "a123"

[[1]]$Destinations
[1] "des1" "des2" "des3"

[[1]]$Days
[1] 6 3 2

[[1]]$Transport
[1] "flight" "car"    "car"


Clients with total travel days exceeding 10 :
Client: des 1 | Itinerary ID: a123 | Total Days: 11
>
```

Ex:No: 11 **Date:**	**Car Inventory Management Using Data Frame and Factor**

Problem Description:

A car dealership wants to maintain an **inventory of cars**, including:

- Car Model (string)
- Registration Number (string)
- Price (numeric)
- Availability (logical: TRUE/FALSE)
- Car Type (categorical: Sedan, SUV, Hatchback)

The dealership wants to:

1. Store car data in a **data frame**.
2. Represent **Car Type as a factor**.
3. **Search cars** based on **availability** and **type**.

All data should be **entered by the user dynamically**.

Objective:

To use data frames and factors in R to store heterogeneous car inventory data, categorize car types, and search for cars based on availability and type.

Sample Input
Enter number of cars: 3 Enter details for Car 1 : Enter car model: Volkswagen Enter registration number: V123 Enter price: 2000000 Is the car available? (TRUE/FALSE): TRUE Enter car type (Sedan/SUV/Hatchback): Sedan Enter details for Car 2 : Enter car model: Ford Enter registration number: F123 Enter price: 1500000 Is the car available? (TRUE/FALSE): TRUE Enter car type (Sedan/SUV/Hatchback): SUV Enter details for Car 3 : Enter car model: BMW Enter registration number: B123 Enter price: 3000000 Is the car available? (TRUE/FALSE): FALSE Enter car type (Sedan/SUV/Hatchback): Hatchback

```r
# Car Inventory Management Using Data Frame and Factor

# User input number of cars
num_cars <- as.numeric(readline(prompt = "Enter number of cars: "))

# Initialize empty vectors
Models <- c()
RegNos <- c()
Prices <- c()
Available <- c()
CarType <- c()

# Input car details
for (i in 1:num_cars) {
  cat("\nEnter details for Car", i, ":\n")

  model <- readline(prompt = "Enter car model: ")
  regno <- readline(prompt = "Enter registration number: ")
  price <- as.numeric(readline(prompt = "Enter price: "))
  avail <- readline(prompt = "Is the car available? (TRUE/FALSE): ")
  type <- readline(prompt = "Enter car type (Sedan/SUV/Hatchback): ")

  Models <- c(Models, model)
  RegNos <- c(RegNos, regno)
  Prices <- c(Prices, price)
  Available <- c(Available, as.logical(avail))
  CarType <- c(CarType, type)
}

# Convert CarType to factor
CarType <- factor(CarType, levels = c("Sedan", "SUV", "Hatchback"))

# Create data frame
car_df <- data.frame(
  Model = Models,
  Registration = RegNos,
  Price = Prices,
  Available = Available,
  Type = CarType
)

# Display complete car inventory
cat("\nCar Inventory:\n")
print(car_df)

# Search cars by availability
search_avail <- as.logical(readline(prompt = "\nEnter availability to search (TRUE/FALSE)
result_avail <- car_df[car_df$Available == search_avail, ]

cat("\nCars based on availability:\n")
print(result_avail)

# Search cars by type
search_type <- readline(prompt = "\nEnter car type to search (Sedan/SUV/Hatchback): ")
result_type <- car_df[car_df$Type == search_type, ]

cat("\nCars based on type:\n")
print(result_type)
```

```
Enter number of cars: 2

Enter details for Car 1 :
Enter car model: ford
Enter registration number: f123
Enter price: 1500000
Is the car available? (TRUE/FALSE): TRUE
Enter car type (Sedan/SUV/Hatchback): SUV

Enter details for Car 2 :
Enter car model: BMW
Enter registration number: B123
Enter price: 3000000
Is the car available? (TRUE/FALSE): FALSE
Enter car type (Sedan/SUV/Hatchback): Hatchback

Car Inventory:
  Model Registration   Price Available      Type
1  ford         f123 1500000      TRUE       SUV
2   BMW         B123 3000000     FALSE Hatchback

Enter availability to search (TRUE/FALSE): TRUE

Cars based on availability:
  Model Registration   Price Available Type
1  ford         f123 1500000      TRUE  SUV

Enter car type to search (Sedan/SUV/Hatchback): SUV

Cars based on type:
  Model Registration   Price Available Type
1  ford         f123 1500000      TRUE  SUV
>
```